

Instituto Politécnico Nacional

Escuela Superior de Cómputo

ESCOM

Trabajo Terminal I

Desarrollo de un General Game Player (GGP) utilizando un lenguaje de descripción de reglas para juegos de información perfecta

2027-A178

Ingeniería en Sistemas Computacionales

Presentan

Sánchez Catarino Edgar
Linares Villegas Gustavo

Director

Dr. en C. Enrique Escalante Notario



2026

INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO
SUBDIRECCIÓN ACADÉMICA

No. de TT: 2027-A178 12 de mayo de 2026

Documento técnico

Desarrollo de un General Game Player (GGP) utilizando un lenguaje de descripción de reglas para juegos de información perfecta

2027-A178

Presentan

Sánchez Catarino Edgar

Linares Villegas Gustavo

Director

Dr. en C. Enrique Escalante Notario

RESUMEN

Este Trabajo Terminal propone el diseño e implementación de un General Game Player (GGP) orientado a juegos discretos, deterministas, por turnos y de información perfecta. La propuesta busca desacoplar las reglas particulares de cada juego del motor de decisión del agente mediante un lenguaje de descripción externo. A partir de dicho lenguaje, el sistema deberá interpretar la estructura del juego, construir una representación interna, generar movimientos legales, aplicar transiciones, reconocer estados terminales y seleccionar acciones con algoritmos generales de búsqueda. El proyecto se valida con Gato como caso funcional completo y con escenarios controlados de Ajedrez para analizar la capacidad de generalización del enfoque sin modificar el núcleo algorítmico del agente.

Palabras clave: General Game Playing, lenguaje de descripción de juegos, inteligencia artificial, juegos de información perfecta, árboles de búsqueda, Monte Carlo Tree Search.

Advertencia

"Este documento contiene información desarrollada por la Escuela Superior de Cómputo del Instituto Politécnico Nacional, a partir de datos y documentos sujetos a derechos de propiedad. Por lo tanto, su uso queda restringido a las aplicaciones que explícitamente se convengan."

Cualquier aplicación no convenida exime a la Escuela Superior de Cómputo de responsabilidad técnica y podrá dar lugar a las consecuencias legales que, para tal efecto, correspondan.

Para obtener información adicional sobre este reporte técnico, podrá dirigirse a:

La Subdirección Académica de la Escuela Superior de Cómputo del Instituto Politécnico Nacional.

Índice general

Índice de figuras

Índice de tablas

Glosario

Término	Definición
Acción legal	Movimiento permitido para un jugador en un estado específico del juego, de acuerdo con las reglas declaradas en la descripción formal del dominio.
Agente inteligente	Sistema computacional capaz de percibir un entorno, razonar sobre sus posibles acciones y seleccionar una decisión con base en un objetivo, recompensa o criterio de utilidad.
Árbol de juego	Representación de las posibles secuencias de estados y acciones de un juego. Cada nodo representa un estado y cada arista representa una acción que conduce a un nuevo estado.
Descripción de juego	Especificación externa que define jugadores, estado inicial, acciones legales, transiciones, estados terminales y recompensas sin codificar directamente esas reglas dentro del motor del agente.
Estado	Configuración completa de un juego en un instante determinado. Debe contener la información necesaria para determinar acciones legales, transiciones y condiciones terminales.
Estado terminal	Estado en el cual la partida termina y se asigna una recompensa o utilidad a cada jugador.
Función heurística	Función de evaluación que estima qué tan favorable es un estado cuando no resulta viable explorar el árbol de juego hasta estados terminales.
General Game Player	Agente capaz de recibir una descripción formal de un juego y jugarlo sin que su motor de decisión contenga reglas específicas programadas para ese dominio.
Información perfecta	Propiedad de los juegos en los que todos los jugadores conocen el estado completo del juego y no existen reglas ocultas para alguno de ellos.
Intérprete	Componente encargado de analizar el archivo de reglas, validar su sintaxis y semántica, y construir una representación interna que pueda consultar el motor del agente.
Lenguaje de descripción de juegos	Lenguaje formal o de dominio específico utilizado para expresar reglas de juegos de manera declarativa y separada del código del motor.
Motor de decisión	Módulo encargado de explorar estados posibles y seleccionar una acción mediante algoritmos generales de búsqueda o simulación.
Poda alfa-beta	Optimización del algoritmo Minimax que evita explorar ramas del árbol de juego que no pueden cambiar la decisión final.

Continúa en la siguiente página...

Término	**Definición**
Recompensa	Valor asignado a un jugador, generalmente al alcanzar un estado terminal, que indica el resultado o utilidad de la partida para ese jugador.
Transición	Cambio de un estado a otro como resultado de aplicar una acción legal.

Abreviaturas

Abreviatura	Significado
AI	Artificial Intelligence (Inteligencia Artificial)
DSL	Domain-Specific Language (Lenguaje de Dominio Específico)
GDL	Game Description Language (Lenguaje de Descripción de Juegos)
GGP	General Game Playing o General Game Player, según el contexto
IA	Inteligencia Artificial
MCTS	Monte Carlo Tree Search (Búsqueda de Árbol Monte Carlo)
RL	Reinforcement Learning (Aprendizaje por Refuerzo)
TT	Trabajo Terminal
UCT	Upper Confidence Bounds applied to Trees

Capítulo 1

Introducción

Capítulo 2

Estado del arte

Capítulo 3

Marco Teórico

3.1. Inteligencia artificial en juegos

Los juegos han sido utilizados de manera recurrente como dominio de prueba para la inteligencia artificial porque ofrecen entornos con reglas explícitas, objetivos medibles y resultados verificables. Desde los primeros trabajos de Arthur Samuel sobre aprendizaje automático aplicado al juego de damas **Samuel1959**, los juegos han permitido estudiar problemas de búsqueda, planeación, toma de decisiones, aprendizaje y evaluación de estrategias.

Una razón por la cual los juegos son relevantes para la inteligencia artificial es que concentran, en un entorno controlado, problemas que también aparecen en dominios más amplios: seleccionar acciones bajo restricciones, anticipar consecuencias, competir con otros agentes, administrar recursos computacionales y tomar decisiones sin explorar por completo todas las posibilidades. En juegos simples, como Gato, es posible recorrer una parte importante del espacio de estados. En juegos más complejos, como Ajedrez, el número de estados y secuencias posibles crece de forma combinatoria, por lo que se requieren estrategias de búsqueda y evaluación.

Históricamente, muchos sistemas exitosos de inteligencia artificial para juegos han sido motores especializados. Estos sistemas alcanzan alto desempeño porque incorporan conocimiento particular del dominio: reglas programadas para un juego, funciones de evaluación ajustadas a sus piezas o posiciones, representaciones hechas a la medida y optimizaciones diseñadas para un tablero específico. Aunque este enfoque puede producir agentes fuertes en un juego determinado, también genera arquitecturas rígidas. Un programa diseñado para jugar Ajedrez no puede, por ese solo hecho, jugar Damas, Go o Gato. La limitación no se encuentra en su capacidad de cálculo, sino en que su conocimiento está unido al juego para el que fue construido.

El problema anterior motiva el estudio de agentes capaces de operar sobre más de un juego sin reprogramar su motor. Esta línea de investigación se conoce como General Game Playing y busca mover parte del trabajo intelectual desde el programador hacia el agente: el sistema no debe depender de que el desarrollador codifique manualmente cada regla, sino de su capacidad para interpretar una descripción formal del juego y razonar sobre ella.

3.2. Juegos de información perfecta

Un juego de información perfecta es aquel en el que todos los jugadores conocen el estado completo del juego cuando toman una decisión. No existen cartas ocultas, información privada, azar no observado ni reglas desconocidas para alguno de los participantes. Ejemplos clásicos de este tipo de juegos son Gato, Ajedrez y Damas. En ellos, el jugador puede ver la configuración actual y determinar qué acciones son legales de acuerdo con las reglas.

Para estudiar estos juegos desde una perspectiva computacional conviene representarlos mediante componentes formales. De manera general, un juego puede describirse a partir de un conjunto de jugadores, un conjunto de estados, un estado inicial, acciones disponibles, una función de transición, estados terminales y valores de utilidad o recompensa. Esta formalización permite que el agente razone con conceptos abstractos en lugar de depender de nombres concretos como "reina", "torre", "X" u "O".

En esta representación, un estado contiene toda la información necesaria para describir una situación del juego. Una acción legal es un movimiento permitido en ese estado. La transición indica cómo cambia el juego después de aplicar una acción. Un estado terminal representa el final de la partida y la utilidad indica qué tan favorable fue el resultado para cada jugador. Esta separación conceptual es fundamental para un General Game Player, porque permite que el motor de decisión opere sobre una interfaz común aun cuando cambien las reglas del juego.

3.3. General Game Playing

Un General Game Player (GGP) es un agente que recibe una descripción formal de un juego y debe jugarlo sin intervención humana ni conocimiento específico preprogramado para ese dominio. Genesereth, Love y Pell definen el área como el estudio de sistemas capaces de aceptar la descripción formal de un juego y jugar efectivamente sin depender de algoritmos diseñados de antemano para juegos particulares **Genesereth2005GGP**.

La diferencia principal entre un motor especializado y un General Game Player se encuentra en la ubicación del conocimiento del juego. En un motor especializado, las reglas y heurísticas suelen estar dentro del código fuente. En un GGP, las reglas se proporcionan como entrada externa. El agente debe interpretarlas, construir una representación interna y usar un proceso general de decisión. Por ello, la generalidad no significa únicamente que el sistema pueda ejecutar dos juegos, sino que el mismo núcleo algorítmico pueda operar al cambiar la descripción del juego.

Este enfoque tiene una implicación directa para el presente Trabajo Terminal: el objetivo no consiste en construir un motor competitivo de Ajedrez comparable con sistemas especializados, sino en diseñar una arquitectura desacoplada. La validación debe demostrar que el sistema puede cargar reglas externas, generar movimientos legales, reconocer estados terminales y seleccionar jugadas bajo un criterio general de búsqueda. La prueba de generalidad se encuentra en cambiar el archivo de reglas sin modificar el motor del agente.

3.4. Modelo formal de juego

El modelo formal de juego en GGP puede entenderse como una máquina de estados. De acuerdo con la descripción general presentada por Genesereth, Love y Pell, un juego finito puede modelarse mediante estados, roles, acciones legales, función de actualización, estado inicial, condiciones terminales y valores de objetivo **Genesereth2005GGP**. Esta visión permite que el juego sea procesado por una computadora como un sistema dinámico.

El conjunto de estados representa todas las configuraciones posibles. Los roles corresponden a los jugadores. Para cada estado, las reglas determinan qué acciones puede ejecutar cada rol. La función de transición toma el estado actual y las acciones realizadas para producir un nuevo estado. El juego continúa hasta alcanzar un estado terminal. En ese punto, se asigna una recompensa a cada jugador.

Para que esta abstracción sea útil en un sistema computacional, el motor del agente debe poder realizar consultas generales. Por ejemplo, necesita obtener el estado inicial, listar acciones legales, aplicar una acción, verificar si un estado es terminal y consultar la recompensa. Estas operaciones pueden expresarse conceptualmente como:

- obtener el estado inicial del juego;
- consultar las acciones legales para un jugador en un estado;
- aplicar una acción y producir un estado sucesor;
- determinar si un estado es terminal;
- calcular la utilidad o recompensa para cada jugador.

La importancia de esta interfaz es que el motor de búsqueda no necesita conocer las reglas concretas del dominio. En lugar de preguntar si un caballo puede moverse en forma de "L" o si una casilla de Gato está vacía, pregunta qué acciones son legales desde el estado actual. El conocimiento específico debe estar en la descripción externa y en la representación interna derivada de ella, no en el algoritmo de decisión.

3.5. Lenguaje de descripción de juegos

Un lenguaje de descripción de juegos permite expresar formalmente las reglas de un juego para que puedan ser interpretadas por un agente. En General Game Playing, el lenguaje clásico es Game Description Language (GDL). GDL fue diseñado para describir juegos discretos mediante reglas lógicas, de modo que un agente pueda conocer el estado inicial, las acciones legales, las transiciones, las condiciones terminales y las recompensas **Love2008GDL**.

GDL utiliza predicados especiales para representar los componentes del juego. Entre los más importantes se encuentran:

- `role`, para declarar jugadores o roles;
- `init`, para indicar hechos verdaderos en el estado inicial;

- `true`, para consultar hechos del estado actual;
- `legal`, para definir acciones permitidas;
- `does`, para representar acciones ejecutadas;
- `next`, para describir hechos del estado siguiente;
- `terminal`, para establecer condiciones de fin de juego;
- `goal`, para asignar valores de recompensa.

Esta forma de representación muestra que GDL no funciona como un archivo simple de configuración. No basta con indicar el tamaño del tablero o el número de jugadores. El lenguaje debe expresar relaciones, condiciones, variables y reglas de inferencia. Por ejemplo, una acción puede ser legal solo si una casilla está vacía y si corresponde el turno del jugador. Esta capacidad declarativa hace que GDL sea potente, pero también introduce complejidad en el diseño del intérprete.

Cropper, Evans y Law explican que las reglas de GGP se describen como programas lógicos y que especifican elementos como estado inicial, movimientos legales, actualización del estado y terminación del juego **Cropper2019IGGP**. Aunque su trabajo se enfoca en aprender reglas a partir de trazas de juego, resulta relevante porque evidencia la complejidad lógica de representar juegos de manera general.

3.6. GDL, Ludii y lenguajes de dominio específico

El uso de GDL clásico no es la única alternativa para describir juegos. Existen otros lenguajes de descripción que buscan equilibrar expresividad, legibilidad y facilidad de implementación. Ludii, por ejemplo, propone un lenguaje de descripción de juegos de mayor nivel, con conceptos más cercanos al diseño de juegos, como tablero, piezas y movimientos **Soemers2024Ludii**.

La comparación entre GDL y Ludii es importante para este Trabajo Terminal porque plantea una decisión arquitectónica: implementar GDL completo, adoptar un subconjunto de GDL o diseñar un lenguaje de dominio específico (DSL) inspirado en estos enfoques. GDL ofrece una base académica fuerte y una semántica lógica general, pero puede resultar complejo para un prototipo de alcance acotado. Ludii, por su parte, muestra que un lenguaje puede conservar generalidad y al mismo tiempo ser más legible para diseñadores y desarrolladores.

Un DSL propio permitiría concentrarse en los elementos necesarios para el alcance del proyecto: jugadores, estados, acciones, reglas de legalidad, transiciones, condiciones terminales y recompensas. Esta opción debe justificarse formalmente para evitar que el lenguaje se convierta en una colección de casos especiales. El criterio de diseño debe ser que el lenguaje permita describir juegos dentro del alcance definido: discretos, deterministas, por turnos y de información perfecta.

3.7. Intérprete y representación interna

El archivo de reglas no es suficiente por sí mismo para jugar. Se requiere un intérprete que lea la descripción, valide su estructura y construya una representación interna consultable por

el motor del agente. Este proceso suele incluir análisis léxico, análisis sintáctico, validación semántica y construcción de un modelo interno.

El análisis léxico separa el texto en unidades reconocibles. El análisis sintáctico verifica que esas unidades respeten la gramática del lenguaje. La validación semántica revisa que las reglas sean coherentes con el modelo del juego, por ejemplo que las acciones referencien jugadores existentes o que una condición terminal pueda evaluarse. Finalmente, la representación interna organiza esa información para responder consultas durante la partida.

La representación interna debe funcionar como puente entre el lenguaje textual y los algoritmos de decisión. Una vez construido el modelo, el motor no debería procesar directamente cadenas de texto, sino estructuras que permitan evaluar legalidad, transiciones y recompensas. Esta separación mejora la mantenibilidad: en principio, el lenguaje textual podría cambiar sin modificar por completo el motor de búsqueda, siempre que se preserve la interfaz conceptual del juego.

En implementaciones avanzadas de GGP, las reglas también pueden compilarse en estructuras más eficientes. Goldwaser y Thielscher mencionan que una descripción GDL puede convertirse en una red proposicional, o PropNet, con nodos lógicos, nodos de entrada, salida y transición **Goldwaser2020DRLGGP**. Esta idea es relevante porque interpretar reglas desde cero en cada nodo de búsqueda puede ser costoso. Aunque implementar una PropNet completa puede quedar fuera de una primera versión, el concepto ayuda a entender por qué la eficiencia de la representación interna es tan importante como el algoritmo de búsqueda.

3.8. Árboles de juego y explosión combinatoria

Una vez que el sistema puede generar acciones legales y aplicar transiciones, se puede construir un árbol de juego. La raíz del árbol representa el estado actual. Cada rama representa una acción legal y cada nodo hijo representa el estado resultante. Al repetir este proceso, el agente puede anticipar posibles futuros y comparar las consecuencias de distintas decisiones.

El problema es que el número de nodos crece rápidamente. Si un estado tiene un factor de ramificación promedio de b acciones y se explora hasta una profundidad d, el número de nodos puede crecer en el orden de b^d. En juegos pequeños, este crecimiento puede ser manejable. En juegos complejos, explorar todas las posibilidades resulta impracticable.

La explosión combinatoria obliga a limitar la búsqueda. El agente debe decidir cuánto explorar, qué ramas priorizar y cómo evaluar estados no terminales. Aquí aparecen algoritmos como Minimax, poda alfa-beta y Monte Carlo Tree Search, así como funciones heurísticas que estiman la calidad de un estado sin llegar necesariamente al final de la partida.

3.9. Minimax

Minimax es un algoritmo clásico para juegos adversariales de dos jugadores. Su principio básico consiste en suponer que un jugador intenta maximizar su utilidad mientras el oponente intenta minimizarla. En un árbol de juego, los niveles se alternan entre decisiones del jugador MAX y decisiones del jugador MIN.

Si el árbol puede explorarse hasta estados terminales, Minimax puede seleccionar una acción óptima bajo el supuesto de que ambos jugadores actúan racionalmente. El valor de un estado

terminal se obtiene directamente de la utilidad del juego. El valor de un estado no terminal se calcula propagando hacia arriba los valores de sus sucesores: MAX elige el mayor valor disponible y MIN elige el menor.

En la práctica, muchos juegos no permiten explorar hasta el final. Por ello, se establece una profundidad máxima y se evalúan los estados frontera mediante una función heurística. Esta adaptación permite aplicar Minimax en juegos más grandes, pero introduce una pregunta central para GGP: cómo evaluar estados sin usar conocimiento específico del juego.

3.10. Poda alfa-beta

La poda alfa-beta es una optimización de Minimax. No cambia la decisión final cuando se aplica bajo las mismas condiciones de evaluación, pero evita explorar ramas que no pueden modificar el resultado. Para ello mantiene dos valores: alfa, que representa la mejor alternativa encontrada por MAX, y beta, que representa la mejor alternativa encontrada por MIN.

Cuando el algoritmo detecta que una rama no puede producir una decisión mejor que otra ya conocida, deja de explorarla. De esta manera, reduce el número de nodos visitados y permite alcanzar mayor profundidad con el mismo presupuesto computacional. La efectividad de la poda depende del orden en que se exploran las acciones; si las mejores acciones se revisan temprano, se pueden descartar más ramas.

Para este Trabajo Terminal, alfa-beta es relevante porque permite comparar dos ideas: la decisión producida por Minimax y la eficiencia lograda al podar ramas. En una evaluación experimental, métricas como nodos explorados, tiempo de decisión y profundidad alcanzada pueden mostrar si la optimización mejora el rendimiento sin alterar la legalidad ni la coherencia de las jugadas.

3.11. Funciones heurísticas generales

Una función heurística estima qué tan favorable es un estado cuando no es posible explorar el árbol hasta el final. En motores especializados, las heurísticas suelen incorporar conocimiento del dominio. En Ajedrez, por ejemplo, puede asignarse valor a piezas o evaluar control del centro. Sin embargo, esas reglas no son generales. Si el agente conoce el valor de una reina, entonces contiene conocimiento específico de Ajedrez.

El reto en General Game Playing es construir evaluaciones que no dependan de la semántica particular de un juego. Una heurística general puede considerar propiedades estructurales, como movilidad, cantidad de acciones legales, proximidad a estados terminales, cambios de recompensa o resultados obtenidos por simulación. Ninguna de estas medidas garantiza por sí sola buen desempeño en todos los juegos, pero pueden servir como aproximaciones dentro de un agente agnóstico al dominio.

Este problema es uno de los puntos más importantes del proyecto. La búsqueda limitada necesita una forma de decidir qué estados parecen mejores, pero la generalidad exige que esa evaluación no esté escrita para un dominio concreto. Por ello, el análisis de heurísticas generales y métodos basados en simulación forma parte central de la fundamentación teórica del motor de decisión.

3.12. Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) es una familia de métodos de búsqueda que estima el valor de las acciones mediante simulaciones. En lugar de depender exclusivamente de una función heurística explícita, MCTS construye progresivamente un árbol de búsqueda y utiliza resultados de partidas simuladas para orientar la exploración **Browne2012MCTS**.

El ciclo general de MCTS se compone de cuatro etapas:

- selección, donde se baja por el árbol eligiendo nodos prometedores;
- expansión, donde se agrega un nuevo nodo;
- simulación, donde se juega desde ese nodo hasta obtener un resultado;
- retropropagación, donde el resultado actualiza los valores de los nodos visitados.

MCTS resulta atractivo para GGP porque puede reducir la dependencia de heurísticas específicas. Si el sistema cuenta con un modelo de juego capaz de generar acciones legales, aplicar transiciones y detectar terminales, entonces puede simular partidas y estimar el valor de acciones a partir de resultados. No obstante, su rendimiento depende de la calidad del modelo interno y del presupuesto de simulación disponible.

3.13. UCT

Una cuestión central en MCTS es decidir qué rama explorar. Si el agente explora solo la acción que hasta ahora parece mejor, puede ignorar alternativas poco visitadas que resultarían superiores. Si explora demasiado al azar, puede perder tiempo en acciones de baja calidad. UCT, o Upper Confidence Bounds applied to Trees, aborda este equilibrio entre explotación y exploración.

UCT combina el valor promedio observado de una acción con un término que favorece acciones menos exploradas. De forma conceptual, una acción es atractiva si ya ha producido buenos resultados o si aún no ha sido visitada lo suficiente. Este criterio permite que MCTS refine progresivamente sus estimaciones sin abandonar por completo la exploración.

En el contexto de General Game Playing, UCT ha sido importante porque ofrece una estrategia relativamente general para seleccionar acciones en árboles de juego sin requerir una heurística escrita manualmente para cada dominio. Goldwaser y Thielscher señalan que agentes exitosos de GGP han usado enfoques basados en búsqueda y heurísticas generales, y que posteriormente se han explorado métodos basados en aprendizaje por refuerzo profundo **Goldwaser2020DRLGGP**.

3.14. Aprendizaje por refuerzo y aprendizaje profundo

El aprendizaje por refuerzo estudia agentes que aprenden mediante interacción con un entorno. En cada paso, el agente observa un estado, ejecuta una acción, recibe una recompensa y transita a un nuevo estado. A diferencia del aprendizaje supervisado, no siempre existe una respuesta correcta inmediata para cada estado; el agente debe aprender a partir de recompensas que pueden aparecer al final de una secuencia de acciones.

En juegos, el aprendizaje por refuerzo puede utilizar resultados como victoria, derrota o empate para ajustar una política de decisión. Cuando se combina con redes neuronales profundas, se habla de aprendizaje por refuerzo profundo. Este enfoque permite aproximar funciones complejas de valor o política, pero suele requerir muchos datos, entrenamiento intensivo y representaciones adecuadas del estado.

Para un GGP, el aprendizaje por refuerzo profundo plantea una dificultad adicional: el agente no solo debe aprender a jugar, sino operar sobre juegos descritos externamente. Esto implica que la representación del estado y de las acciones debe ser suficientemente general. Por esta razón, el aprendizaje profundo se considera en este trabajo principalmente como antecedente y línea de investigación avanzada, no como requisito central de implementación.

3.15. AlphaZero como antecedente moderno

AlphaZero representa un antecedente importante porque combinó autojuego, aprendizaje por refuerzo profundo y búsqueda Monte Carlo para alcanzar alto desempeño en Ajedrez, Shogi y Go **Silver2018AlphaZero**. Su arquitectura utiliza una red neuronal para estimar una política de acciones y un valor de estado, mientras MCTS refina la selección de movimientos mediante búsqueda.

Sin embargo, AlphaZero no resuelve por sí mismo el problema central de este Trabajo Terminal. Aunque emplea un algoritmo general en más de un juego, requiere simuladores y representaciones adecuadas para cada dominio. En un GGP, el reto principal es distinto: interpretar reglas externas, construir el modelo del juego y permitir que el motor de decisión opere sin conocimiento específico codificado para cada caso.

Goldwaser y Thielscher extienden ideas de aprendizaje por refuerzo profundo al contexto de GGP y muestran que existe una línea de investigación que busca unir descripciones formales de juegos con métodos inspirados en AlphaZero **Goldwaser2020DRLGGP**. Para el alcance de este Trabajo Terminal, estas ideas sirven como referencia del estado del arte y ayudan a delimitar el proyecto: se busca una arquitectura general e interpretable, no replicar un sistema de entrenamiento profundo a gran escala.

3.16. Síntesis del marco teórico

El desarrollo de un General Game Player requiere integrar varias áreas de la inteligencia artificial y de las ciencias computacionales. Primero, se necesita una representación formal del juego que separe estados, acciones, transiciones, terminalidad y recompensas. Segundo, se requiere un lenguaje de descripción o DSL capaz de expresar reglas sin codificarlas directamente en el motor. Tercero, es necesario un intérprete que transforme esa descripción en una representación interna consultable. Finalmente, el agente debe seleccionar acciones mediante algoritmos generales de búsqueda o simulación.

La cadena conceptual del proyecto puede resumirse como:

reglas externas → intérprete → representación interna → acciones legales → árbol de juego → decisión.

La aportación del trabajo no radica en construir el mejor jugador de un juego particular, sino en demostrar una arquitectura desacoplada. El motor debe mantenerse estable cuando cambia la descripción del juego. Por ello, Gato sirve como caso funcional completo para validar el flujo de reglas, estados y terminalidad, mientras Ajedrez funciona como caso controlado de mayor complejidad. Esta delimitación permite estudiar generalidad operacional sin confundir el prototipo con un motor especializado de competencia.

Capítulo 4

Análisis

Capítulo 5

Diseño

Capítulo 6

Desarrollo

Capítulo 7

Pruebas

Capítulo 8

Conclusiones y trabajo a futuro

Anexos

8.1. Cronogramas